Technical Requirements

AstroIllustrator | Consolidated specification v2.1 | 22 October 2026

1. Architecture and stack

Keep three .NET projects: AstroIllustrator.Api, .Core, .Infrastructure; and one separate Python renderer package. Core has no infrastructure dependencies; Api composes services; Infrastructure adapts Python process, EF Core and optional AI provider. Use supported .NET LTS, ASP.NET Core Web API, EF Core/SQLite for metadata; Python with Astropy and NumPy for celestial computations; Matplotlib/SVG tooling for precise diagrams. Validate plotting library API and commercial licenses in a proof of concept.

2. Generalized rendering contracts

Expose a discriminated illustration request with illustrationType = SkyChart | ScientificDiagram | ArtisticIllustration and a type-specific payload; avoid a giant nullable DTO. Use explicit enums and schema versions. SkyChartRequest includes sceneType, constellationCodes or asterismCode, observer, timeUtc, projection, framing, layers, preset, export. ScientificDiagramRequest includes diagramType, parameters, language, preset, export. CompositeRequest references immutable source render and background asset.

3. Astronomical accuracy

Use documented star catalog snapshots with epoch/reference frame, magnitude, proper motion if needed, and provenance. Use vetted IAU constellation identifiers and boundaries; line-segment drawings are convention-specific, so record their source and license. For time-dependent sky charts, transform ICRS/catalog coordinates into topocentric AltAz using observer location and UTC time; state atmospheric refraction assumptions, horizon model, elevation and whether obstructions are modeled. Use a projection appropriate to each scene; sky chart geometry must be reproducible and must not be distorted by decorative compositing. Never infer real star coordinates from AI imagery.

4. Diagram geometry

Build template-based deterministic diagrams for altitude/azimuth, celestial sphere, zenith/nadir, Polaris/latitude and equatorial coordinates. Generate angle arcs from the same numeric values used in labels; separate pedagogical schematic mode from physically computed observation mode. A star at 45 degrees altitude must be geometrically depicted at 45 degrees relative to its local horizon reference. Azimuth convention: 0 degrees north, 90 east, 180 south, 270 west; document conventions in every template. For Polaris, north celestial pole altitude approximates observer geographic latitude for a level astronomical horizon; exact Polaris altitude requires observation time and location. Allow an explicitly marked approximate teaching diagram when time is absent.

5. Rendering, export and validation

Render scientific content as independent vector layers: horizon/axes, stars, line segments, angles, labels, annotations. Produce SVG and PNG; convert to PDF using a vector-preserving path when possible. Validate angle bounds, coordinates, allowed templates, catalog availability, file sizes, projection, label collisions and required time/location. Use reference charts, unit tests for transforms, numeric tolerances and snapshot visual regression tests.

6. AI integration boundary

LLM receives prompt and typed schema; returns structured IllustrationRequest only. Server validates it, resolves ambiguous fields or asks for missing observation time if exact positioning is requested. Optional background model generates star-free scenic imagery. Composite under a fixed astronomical overlay. Store model/provider/version, prompt, asset rights and generation cost. AI outage must not break manual rendering. Optional RAG may ground educational captions in approved sources with citations and editorial review. No vector database or multi-agent framework in MVP.

7. Nonfunctional requirements

Local-first, deterministic renders, versioned catalogs, secure process invocation without shell concatenation, bounded CPU/memory and file paths, cancellation, timeouts, structured logging, retry only transient failures. API contract tests; Python renderer unit tests; golden image comparisons; packaging guidance for Windows; attribution and commercial-use review for every dataset and generated background. Document control: This version supersedes earlier 1.0 and 1.1 scope descriptions. Keep requests, manifests, diagrams and AI integrations aligned with this specification.

v2.1 additions: global observer support, time-aware conjunctions and AstroIllustrator naming

Observer and temporal reference model

ObserverLocation: WGS84 latitude [-90,+90], longitude [-180,+180], elevationMeters and optional timeZoneId (IANA preferred for display). Store UTC instants; resolve requested local date/time with daylight-saving ambiguity checks. For sunset-relative requests compute the event at the specified location/date, then add signed offset minutes; handle polar day/night and no-event cases explicitly.

Solar system ephemeris engine

Add Python astronomy/ephemeris modules using validated JPL planetary ephemerides via Skyfield or Astropy (choose one primary path, document kernels, coverage and licenses). Compute topocentric apparent Sun/Moon/planet positions, altitude/azimuth, angular separation and Moon phase/orientation. Account for parallax (especially Moon), epoch/time scales and configurable refraction. No ephemeris outside supported kernel dates.

Twilight and event semantics

Define sunrise/sunset convention (apparent solar upper limb, refraction and horizon assumption) and civil/nautical/astronomical twilight solar depression thresholds (-6/-12/-18 degrees). Distinguish geometrical and apparent altitude. Event detection finds minima of angular separation over a bounded interval, with sampling and numerical refinement; distinguish geocentric conjunction from observer-visible closest approach.

Sky rendering and observability

Azimuth measured from true north clockwise (west = 270 degrees). Render western horizon with accurate altitude-azimuth projection and documented field of view. Compute object altitude, Sun depression, optional extinction/airmass and brightness; do not claim guaranteed naked-eye visibility. Treat scenic twilight gradient as aesthetic unless physically modeled; lock ephemeris-derived celestial layers.

Scientific note: illustrative dates and prompts are examples, not verified event predictions. All published charts require computed ephemerides and editorial review.